

Week- 5

# Backend Engineering

Pawan Dhanjwari  
**Launchpad**

# Authentication and Authorization

# Agenda

- What is authentication and authorization ?
- Understanding tokens
- Understanding login and signup - Token based authentication using mysql
- Java dependencies - bcrypt
- Role based authentication and authorization for LMS
- OAuth 2.0
- Question and answers

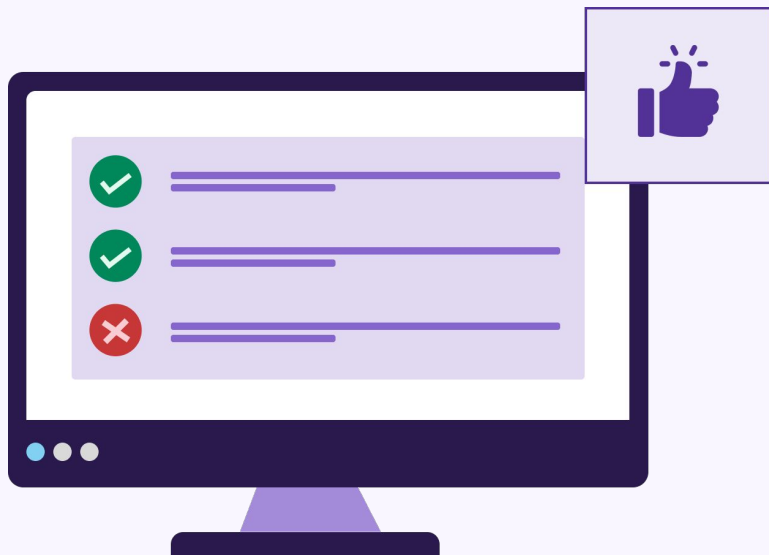
# What is authentication



Authentication:

Confirms Users are  
who they say they are

# What is authorization



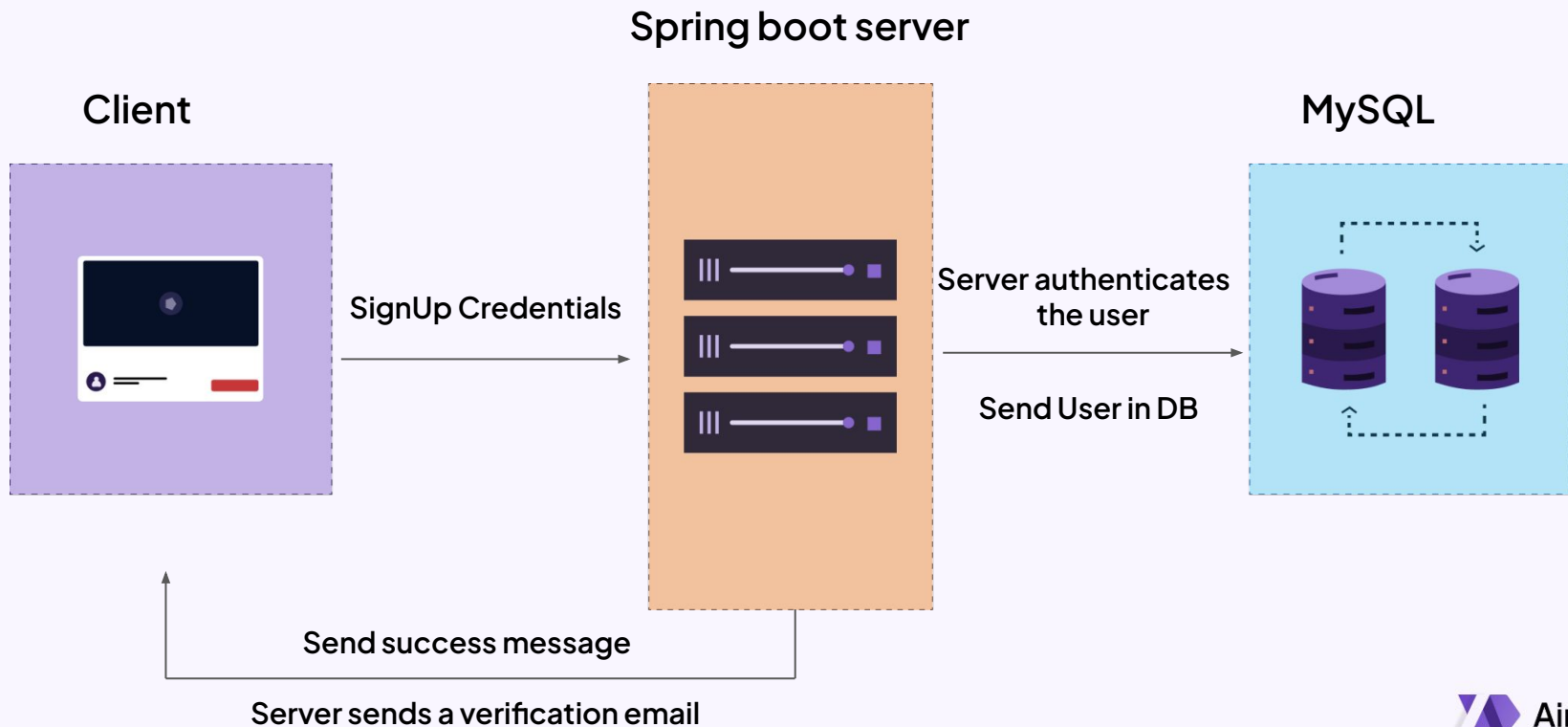
## Authorization:

Give users permission to access a resource.

# What is Authentication & Authorization

|                  | Authentication                                                                                                                                                                   | Authorization                                                                                                                                                                                                                   |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PURPOSE          | <ul style="list-style-type: none"><li>• Verifies user identity</li></ul>                                                                                                         | <ul style="list-style-type: none"><li>• Permits access to resources</li></ul>                                                                                                                                                   |
| REQUIREMENTS     | <ul style="list-style-type: none"><li>• Identity credentials based on knowledge, possession, and/or inference</li><li>• Authentication solution</li></ul>                        | <ul style="list-style-type: none"><li>• Authenticated identity and access control policies</li><li>• Authorization solution</li></ul>                                                                                           |
| RESPONSIBILITIES | <ul style="list-style-type: none"><li>• Network security staff determine which factors to adopt.</li><li>• Users provide authentication factors when requesting access</li></ul> | <ul style="list-style-type: none"><li>• Leadership sets security strategies</li><li>• Departments and workgroups define access criteria</li><li>• Network security staff implement and maintain access control system</li></ul> |

# Auth using MySql and Spring Boot



# Session Based Authentication Flow

Register

Verify  
Registration

Login

Session Based  
Authentication



# Token Based Authentication Flow

Register

Verify  
Registration

Login

Token Based  
Authentication

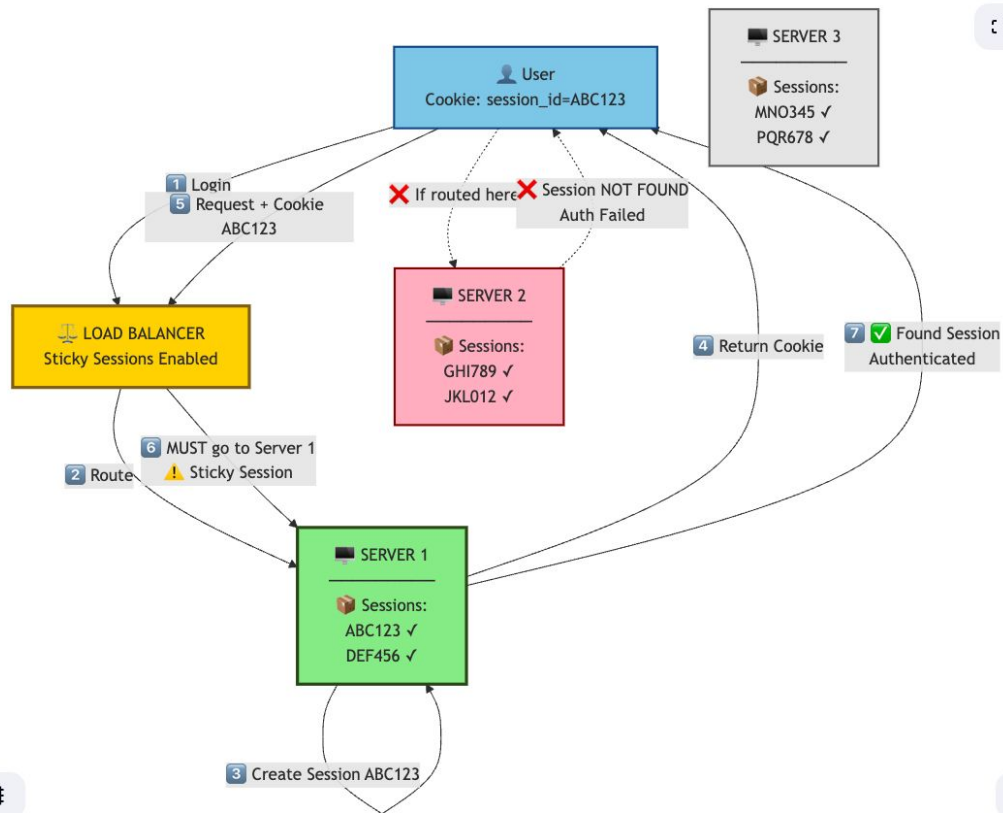
# Session Based Authentication

[Flow chart](#)

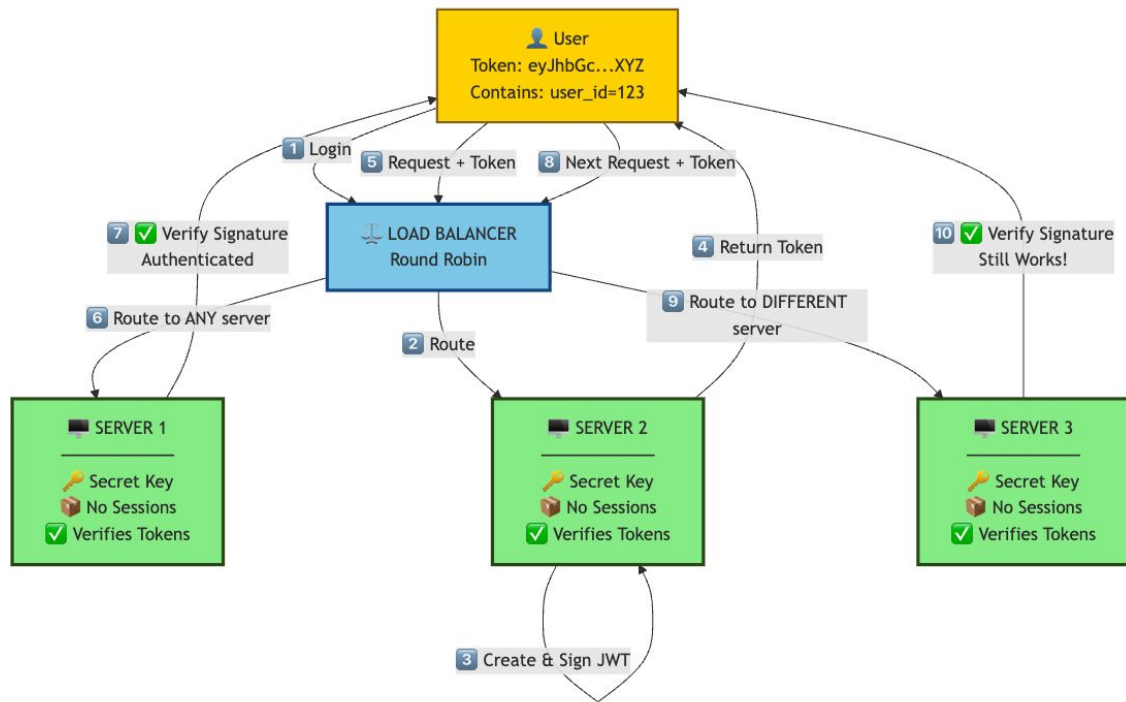
# Token Based Authentication

[Flow chart](#)

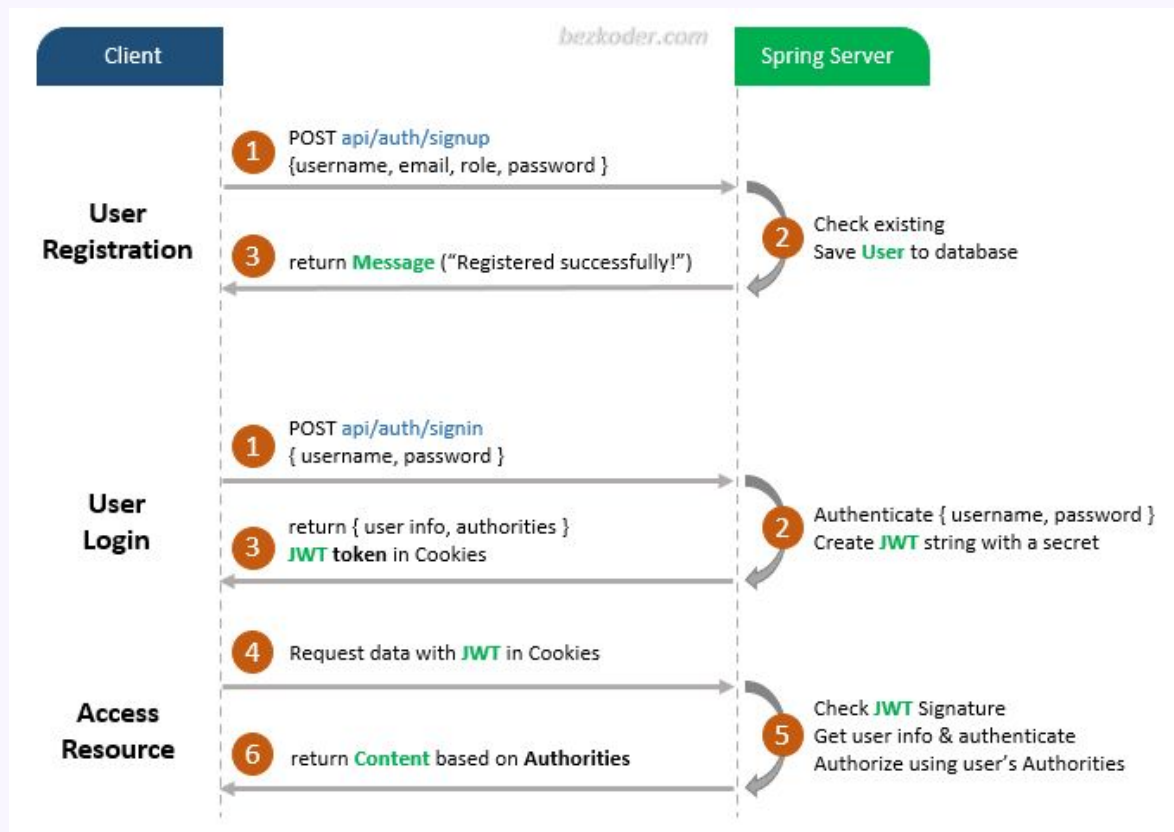
# Stateful Session based authentication



# Stateless Token based authentication



# User Registration and login flow



# Registration flow

## User Registration Form

- **Client:** User fills out the registration form.
- **Action:** The client submits registration details (e.g., username, password, email).

## Send Registration Request

- **Client → Server:** HTTP POST request to `/register` endpoint with registration data.
- **Payload Example:** `{ "username": "user1", "password": "pass123", "email": "user1@example.com" }`.

## Server Validation

- **Server:** Validates the input (e.g., check for existing usernames, validate email format).
- **Decision:**
  - **Valid:** Proceed to user creation.
  - **Invalid:** Return error response to the client.

# Registration flow

## Create User Account

- **Server → Database:** Hashes the password and stores the user details in the database.
- **Response:** Success or error message sent back to the client.

## Send Confirmation Email (Optional)

- **Server:** Sends a confirmation email to the user for email verification (optional).

## Return Response

- **Server → Client:** Sends response with success message or validation errors.



# Add: Encryption vs Hashing (1)

1. **Encryption** is a process of converting data into a secret code so that it can only be read by authorized parties who have the decryption key.
2. **Hashing**, on the other hand, is a one-way process of converting data into a fixed-length string of characters, where the output cannot be reversed back to the original input.

# Add: Encryption vs Hashing (2)

1. **Encryption** can be reversed. Encryption is used to protect data that needs to be decrypted later, such as passwords or sensitive data. In Node.js, encryption can be achieved using various algorithms, such as AES or RSA
2. **Hashing** is a one way process .hashing is used to verify the integrity of data or to store passwords securely. hashing can be achieved using algorithms such as SHA-256 or bcrypt.

# Strong password?

password

bcrypt

\$2b\$10\$gOancHkngDJvrXkW809800

# OAuth 2.0

- Resource Owner
- Resource Server
- Client
- Authorization Server
- Access Token
- Refresh Token

# OAuth 2.0

## 1. Resource Owner

- **Definition:** The user or entity that owns the resource.
- **Role:** Grants permission to the client application to access their resources.

## 2. Resource Server

- **Definition:** The server that hosts the protected resources (e.g., APIs).
- **Role:** Validates access tokens and serves the requested resources to clients.

## 3. Authorization Server

- **Definition:** The server responsible for issuing access tokens to clients after successful authentication and authorization.
- **Role:** Handles user authentication and issues access tokens.

# OAuth 2.0

## 4. Client

- **Definition:** The application requesting access to the resources on behalf of the resource owner.
- **Role:** Requests authorization and uses the access tokens to access resources.

## 5. Access Token

- **Definition:** A token issued by the authorization server that represents the authorization granted to the client.
- **Role:** Used by the client to access protected resources on the resource server.

# OAuth 2.0

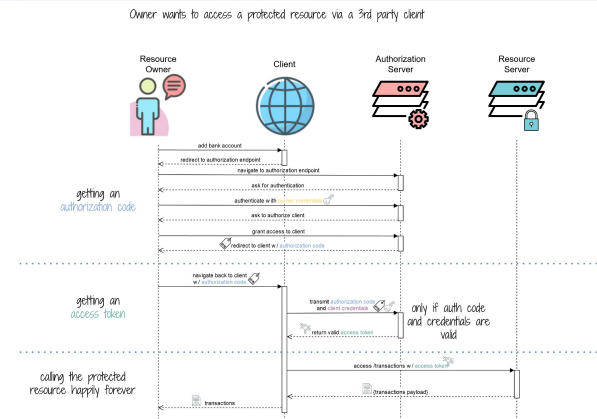
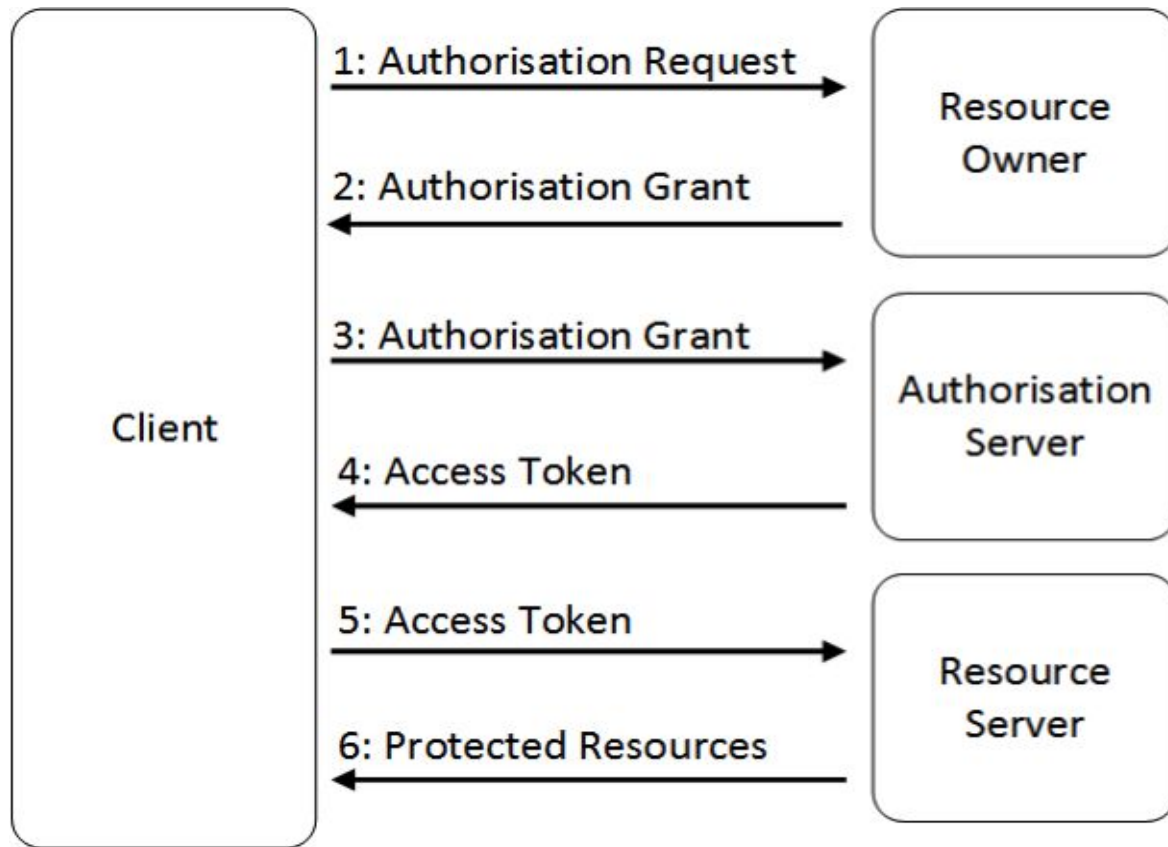
## 6. Refresh Token

- **Definition:** A token used to obtain a new access token when the current access token expires.
- **Role:** Allows the client to maintain access without re-authenticating the user.

## 7. Scope

- **Definition:** A parameter that specifies the level of access requested by the client.
- **Role:** Determines what resources and actions are allowed for the client.

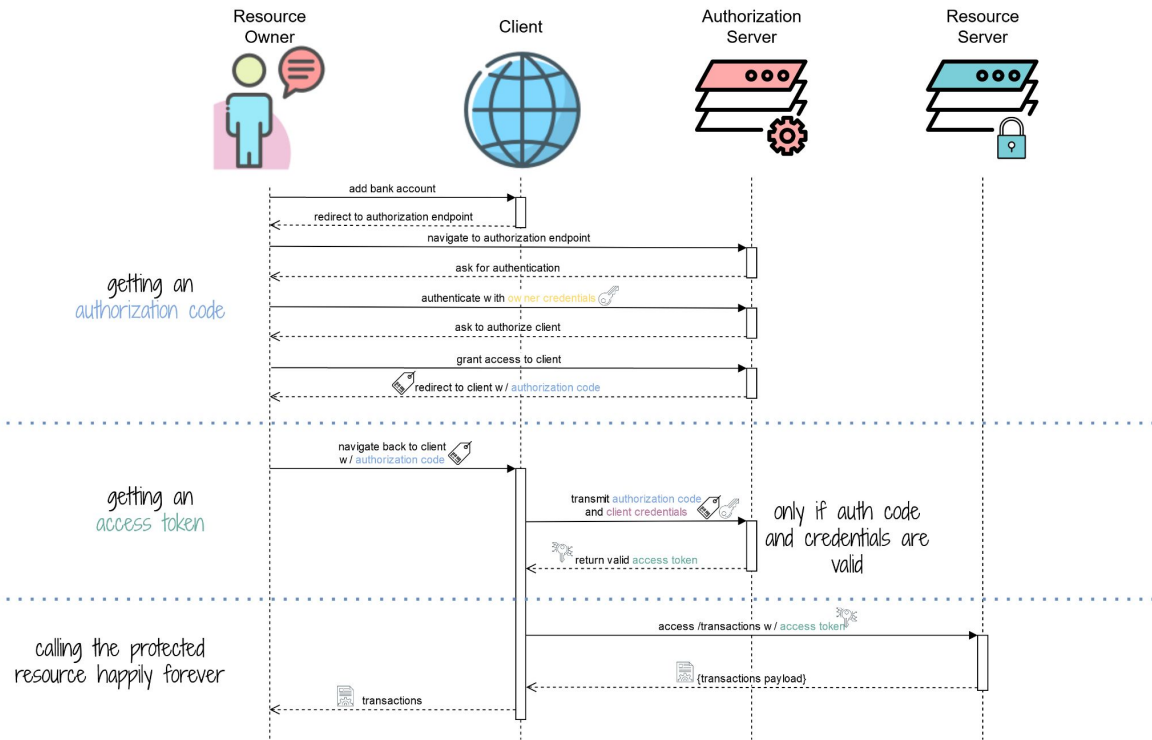
# OAuth 2.0





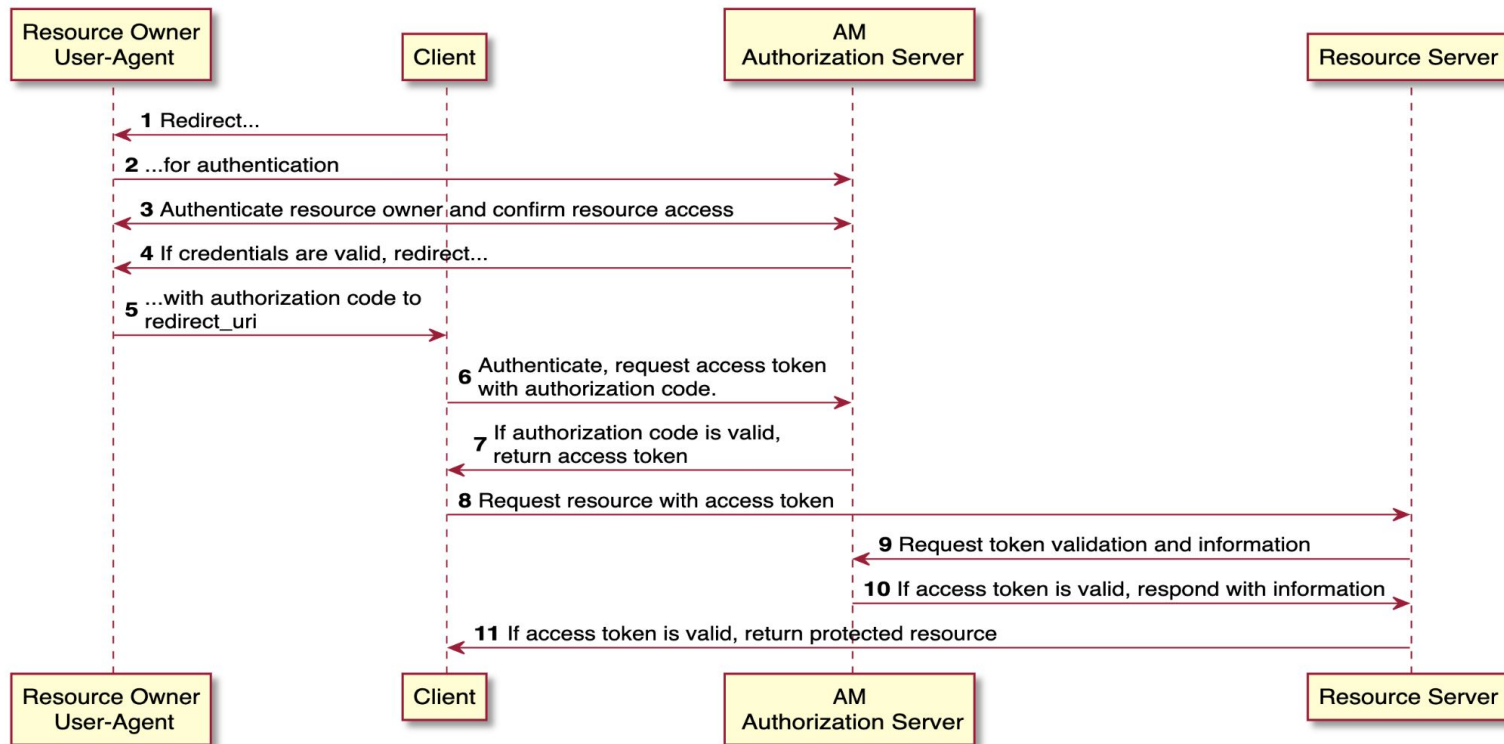
# OAuth 2.0 Flow Diagram

Owner wants to access a protected resource via a 3rd party client

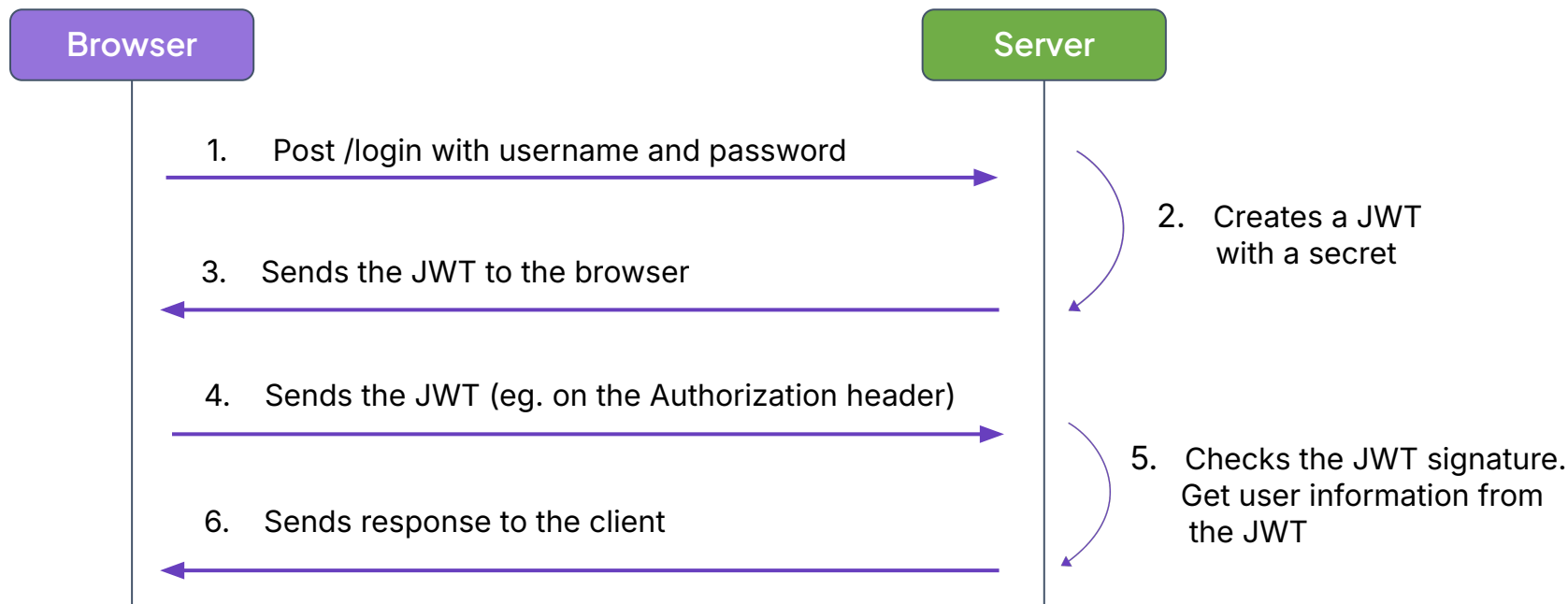


# OAuth 2.0 Flow Diagram

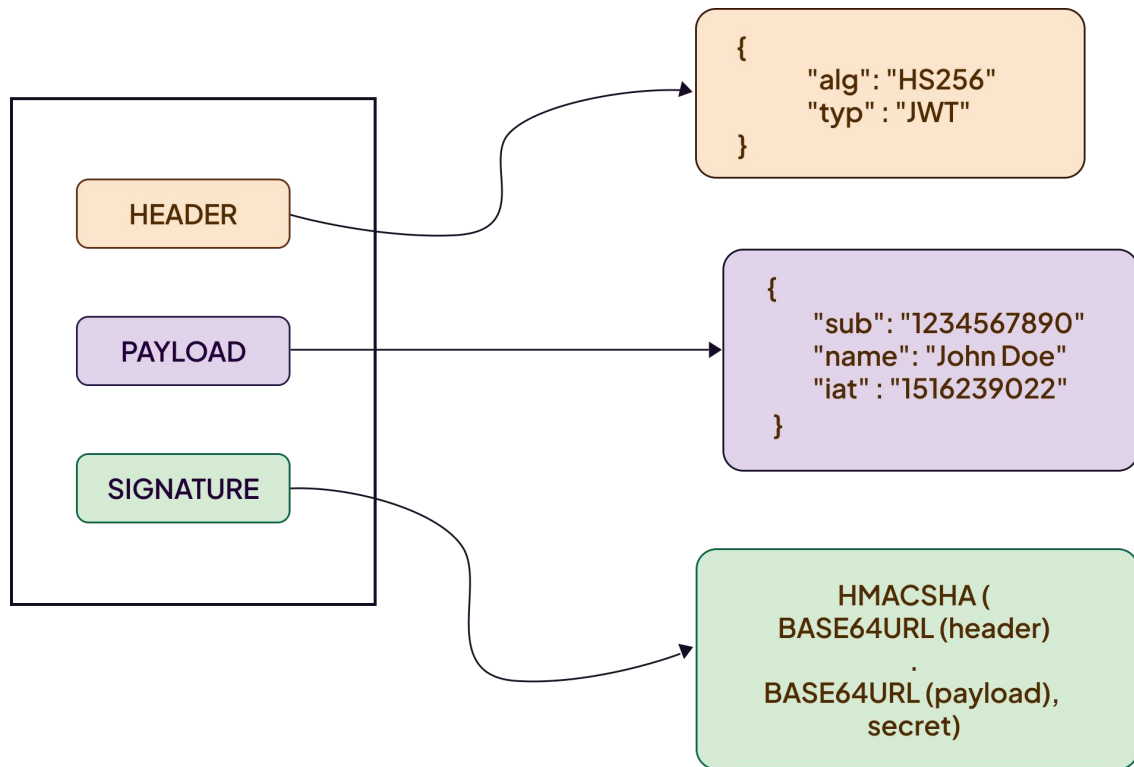
## OAuth 2.0 Authorization Code Grant Flow



# What is JWT based Authentication & Authorization



# Structure of JWT



# OAuth 2.0 playground

<https://developers.google.com/oauthplayground/>

<https://www.oauth.com/playground/>

# Got Questions?

# Thank you